

ЛАБОРАТОРНАЯ РАБОТА № 1

Цветовые модели

Цель работы

Изучить основные цветовые модели RGB, CMYK, HSV, HLS, XYZ и CIELAB (Lab), принципы перехода между цветовыми пространствами, понятие цветового охвата (gamut) и проблему отображения цветов, не входящих в диапазон sRGB. Освоить разработку интерактивного веб-приложения, позволяющего выбирать и изменять цвет, одновременно отображая его представления в трёх заданных цветовых моделях: **RGB, HSV и CMYK**.

Задачи работы

В ходе выполнения лабораторной работы были поставлены следующие задачи:

- изучить назначение и область применения цветовых моделей RGB, CMYK, HSV, HLS, XYZ и Lab;
- рассмотреть преобразования между цветовыми пространствами;
- изучить понятие цветового графика МКО, цветового охвата и выхода цвета за границы отображаемого диапазона;
- реализовать веб-приложение в браузере с одновременным отображением RGB, HSV и CMYK и предупреждением при ограничении цвета до доступного диапазона RGB;
- разместить исходный код в GitHub и подготовить приложение к публикации на статическом хостинге.

Использованные средства разработки

- HTML5 — структура веб-страницы, элементы управления (``input type="range"`, `input type="number"`, `input type="color"```).
- CSS3 — стилизация интерфейса (Flexbox, скруглённые блоки, цветовая индикация).
- JavaScript (ES6+) — реализация математических функций преобразования цветовых моделей, обработка событий, синхронизация интерфейса.
- Git и GitHub — контроль версий и размещение исходного кода.
- Netlify— статический хостинг веб-приложения.

Ход выполнения работы

1. Разработка интерфейса

2.

Было создано веб-приложение «Конвертер цветовых моделей» на чистом HTML/CSS/JavaScript. В интерфейсе сформированы три панели — RGB, HSV и CMYK, а также отдельный блок предварительного просмотра выбранного цвета (`#preview`).

Для каждого параметра цвета реализованы два способа ввода:

- ползунок `input type="range"` — для плавного изменения;
- числовое поле `input type="number"` — для точного числового ввода.

В панели RGB дополнительно добавлен элемент `input type="color"`, использующий системный color picker браузера.

2. Визуальный выбор цвета

Элемент `input type="color"` возвращает выбранный цвет в HEX-представлении (например, `#ff0000`). Из HEX-строки извлекаются RGB-компоненты путём парсинга пар символов:

```
````javascript
const r = parseInt(hex.substr(1, 2), 16);
const g = parseInt(hex.substr(3, 2), 16);
const b = parseInt(hex.substr(5, 2), 16);
````
```

После этого запускается общий механизм пересчёта всех цветовых моделей.

3. Реализация преобразований цветов

В JavaScript реализованы следующие функции преобразования:

- `rgbToHsv(r, g, b)` — перевод RGB в HSV. Вычисляются максимальный и минимальный каналы, разница `d = max - min`. Оттенок `H` определяется по формуле в зависимости от того, какой канал максимален (R, G или B), насыщенность `S = d / max`, яркость `V = max`.
- `hsvToRgb(h, s, v)` — перевод HSV в RGB. Используется промежуточное значение `c = v · s` и `x = c · (1 - |h/60 mod 2 - 1|)`, после чего по диапазону оттенка выбирается одна из шести формул.
- `rgbToCmyk(r, g, b)` — перевод RGB в CMYK. Чёрная компонента `K = 1 - max(R', G', B')`, остальные компоненты вычисляются как `(1 - канал - K) / (1 - K)`.
- `cmykToRgb(c, m, y, k)` — перевод CMYK в RGB по формуле `R = 255 · (1 - C) · (1 - K)`.

4. Автоматический пересчёт

Для синхронизации всех моделей были реализованы три функции-обработчика: ``updateFromRgb()``, ``updateFromHsv()`` и ``updateFromCmyk()``.

При изменении ****RGB**** выполняется последовательность:

- RGB → HSV;
- RGB → CMYK.

При изменении ****HSV**** выполняется последовательность:

- HSV → RGB;
- RGB → CMYK.

При изменении ****CMYK**** выполняется последовательность:

- CMYK → RGB;
- RGB → HSV.

Для защиты от циклических обновлений (когда изменение одного поля вызывает изменение другого, которое снова вызывает первое) введён флаг ``isUpdating``. Пока функция выполняется, другие обработчики игнорируются.

5. Проверка диапазонов и округление

В приложении добавлена функция ``clamp(value, min, max)``, которая ограничивает значения допустимыми диапазонами:

- RGB: 0–255;
- H: 0–360°;
- S, V, C, M, Y, K: 0–100%.

RGB-каналы отображаются целыми числами, HSV и CMYK — также целыми (в процентах и градусах).

6. Предупреждение о выходе за gamut

При преобразовании ****CMYK → RGB**** теоретически возможно получение значений за пределами 0–255 (хотя при корректных входных CMYK 0–100% этого не происходит). nevertheless, в коде реализована проверка: если после вычисления ``cmykToRgb`` хотя бы один канал выходит за диапазон и был обрезан функцией ``clamp``, пользователю показывается предупреждение:

Значение выходит за границы модели — выполнено обрезание (clipping)

Предупреждение отображается 2 секунды и не блокирует дальнейшую работу.

Проблемы, возникшие в работе, и способы решения

1. Циклические обновления интерфейса.

-Причина:при изменении RGB обновлялись HSV и CMYK, а их обработчики, в свою очередь, могли снова запустить пересчёт RGB, создавая бесконечный цикл.

-Решение:введён флаг `'isUpdating'`. Пока одна из функций `'updateFrom*'` выполняется, другие обработчики возвращаются без действий.

2. Некорректное отображение чёрного цвета в HSV.

- Причина: при $V = 0$ (чёрный цвет) оттенок H и насыщенность S математически не определены (деление на ноль в формуле $S = d/\max$).

- Решение: в функции `'rgbToHsv'` добавлена проверка `'max === 0'`, при которой S и H принудительно устанавливаются в 0.

3. Потеря информации при обратном преобразовании CMYK $\rightarrow$ RGB.

- Причина: CMYK — субтрактивная модель, и не все её комбинации однозначно соответствуют RGB-цветам после округления.

- Решение: добавлена функция `'clamp'` и визуальное предупреждение при обрезании значений.

4. Синхронизация color picker с ползунками RGB.

- Причина: элемент `'input type="color"'` принимает только HEX-формат.

- Решение: RGB-компоненты преобразуются в HEX-строку методом `'toString(16).padStart(2, '0')'` и присваиваются полю color picker.

5. Размещение на статическом хостинге.

- Причина: приложение состоит из одного HTML-файла без системы сборки.

- Решение:файл загружен напрямую в GitHub-репозиторий, хостинг Netlify настроен на публикацию корневой папки репозитория.

Результаты работы

В результате выполнения лабораторной работы создано веб-приложение — конвертер цветовых моделей, которое:

- отображает выбранный цвет в RGB, HSV и CMYK;

- позволяет вводить точные значения компонентов через числовые поля;
- позволяет плавно менять компоненты с помощью ползунков;
- позволяет выбирать цвет из палитры браузера (`input type="color"`);
- автоматически пересчитывает другие модели при изменении любого параметра;
- показывает preview текущего RGB-цвета;
- выводит предупреждение при выходе выбранного цвета за пределы допустимого диапазона;
- содержит исходный код в GitHub-репозитории и опубликовано на статическом хостинге Netlify.

Выводы

Практическим результатом стало создание интерактивного веб-приложения, в котором реализованы все основные способы задания цвета и их взаимные преобразования между моделями $\text{CMYK} \leftrightarrow \text{RGB} \leftrightarrow \text{HSV}$.

Для перевода между моделями использовались стандартные математические формулы: через нормализацию RGB-каналов к диапазону $[0, 1]$ и обратно. Прямой перевод $\text{CMYK} \leftrightarrow \text{HSV}$ невозможен — требуется промежуточное пространство RGB, что и было реализовано в приложении.

В ходе работы было выяснено, что:

- при $V = 0$ в HSV цвет всегда чёрный, независимо от значений H и S;
- при $K = 100\%$ в CMYK цвет всегда чёрный, независимо от C, M, Y;
- округление целочисленных значений при обратных преобразованиях может приводить к небольшой потере точности (1–2 единицы), что является нормальным поведением.

Использование чистого JavaScript без внешних фреймворков позволило создать лёгкое приложение, работающее в любом современном браузере без этапа сборки.

.